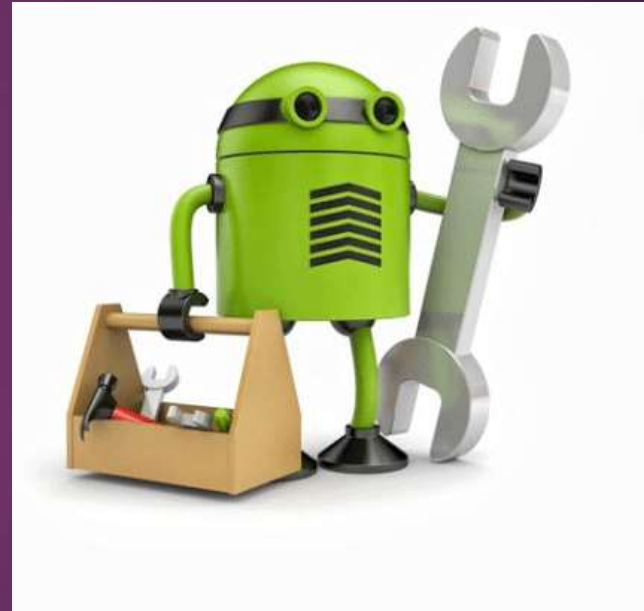


Android Application Testing

TANJINA ISLAM

[linkedin.com/in/tanjinaislam](https://www.linkedin.com/in/tanjinaislam)



Agenda/Outline/Overview

- ❑ Testing Approach
- ❑ Android testing framework
- ❑ Android The Testing API
- ❑ Android testing tools
- ❑ Different automated testing frameworks

Testing Approach

- ❑ UNIT TESTING
- ❑ INTEGRATION TESTING
- ❑ MANUAL TESTING
- ❑ AUTOMATED TESTING

Unit Testing

- ❑ tests only the functionality of a certain component such as a method (function) in a class, with all dependencies mocked up
- ❑ tests tend to be simpler and faster than integration tests
- ❑ also known as local tests
- ❑ run on a local JVM on the development machine instead of the Android Runtime
- ❑ the execution speed of the unit test is very fast compared to tests which require the Android system
- ❑ unit tests are executed against a modified version of the android.jar which allows you to mocking libraries, like Mockito

Unit Testing

□ Tools : JUnit

```
@Test
public void testAddOnePlusOne()
{
    int x = 1;
    int y = 1;
    assertEquals(2, myCalculator.add(x,y));
}
```

Integration Testing

- ❑ tests more than one component and how different pieces of the system work together
- ❑ requires resources like database instances and hardware to be allocated for them
- ❑ test many methods and may interact with dependencies like Databases or Web Services

Unit Testing Vs. Integration Testing

- ❑ Let's, for example, assume a button in an Android activity is used to start another activity. A unit test would determine if the corresponding intent was issued, not if the second activity was started.
- ❑ An integration test would also check if the activity was correctly started.

Manual Testing

- ❑ Requires Human interaction
- ❑ Tool : Human being



Automated Testing Frameworks

- ❑ No Human Interactions
 - ❑ Lots of Frameworks to automate testing
-
- Robotium
 - Espresso
 - uiautomator
 - Robolectric
 - Calabash
 - Appium



Automated Testing Frameworks

- ❑ most popular approaches used for automated Android testing:
 - **Google's Android Testing Framework**- This is the framework included as part of the platform
 - **Robotium** - Black box integration testing for Android
 - **Robolectric** - Unit tests that run outside the emulator making the tests very fast

Android Testing Framework

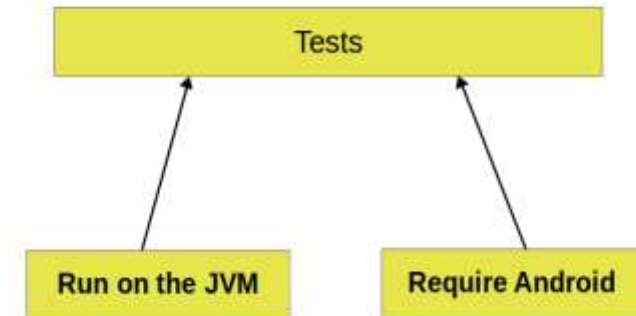
- ❑ An Integral part of the development environment
- ❑ Assist on testing every aspect of application from unit testing to framework



Categories of Android Tests

- ❑ Testing for Android can be classified into:
 - Local tests - tests which can run on the JVM
 - Instrumented tests - tests which require the Android system
- ❑ If possible, you should prefer to use local tests as the test execution is much faster compared to the time required to deploy and run the test on an Android device

Categories of Android tests



Android The Testing API

- The Android testing API is based on the JUnit API and extended with a instrumentation framework and Android-specific testing classes.

- ❑ **JUnit**

- ❑ **Instrumentation**

- ❑ **TestCase Classes**

- AndroidTestCase
- Component-specific test cases
- ApplicationTestCase
- InstrumentationTestCase

- ❑ **Assertion classes**

- ❑ **Mock object classes**

- ❑ **Contexts for testing**

Android Testing Framework

- ❑ Key Features of Testing Framework

- ❑ test suites are based on JUnit

- use plain JUnit to test a class that doesn't call the Android API
 - use Android's JUnit extensions to test Android components

- ❑ test suites are contained in test packages similar to main application packages

- ❑ The SDK also provides

- monkeyrunner, an API for testing devices with Python programs
 - UI/Application Exerciser Monkey, a command-line tool for stress-testing UIs by sending pseudo-random events to a device

Android Testing Tools

- ❑ Testing Support Library

- ❑ This library provides a set of APIs that helps to build and run test code for app
- ❑ The Android Testing Support Library includes the following test automation tools:
 - **AndroidJUnitRunner**: JUnit 4-compatible test runner for Android
 - **Espresso**: UI testing framework; suitable for functional UI testing within an app
 - **UI Automator**: UI testing framework; suitable for cross-app functional UI testing across system and installed apps

Android Testing Tools

❑ Testing Support Library

❑ Monkey

- is a command-line tool
- a program that runs on emulator or device
- generates pseudo-random streams of user events (such as clicks, touches, or gestures, as well as a number of system-level events) and sends them into system
- acts as a stress test on the application, in a random yet repeatable manner
- watches the system under test

Android Testing Tools

- ❑ Testing Support Library

- ❑ monkeyrunner

- provides an API for writing programs that control an Android device or emulator from outside of Android code
 - monkeyrunner tool provides these unique features for Android testing: Multiple device control, Functional testing, Regression testing, Extensible automation
 - The monkeyrunner tool uses Jython
 - Jython allows the monkeyrunner API to interact easily with the Android framework

Android Testing Tools

- ❑ Testing Support Library

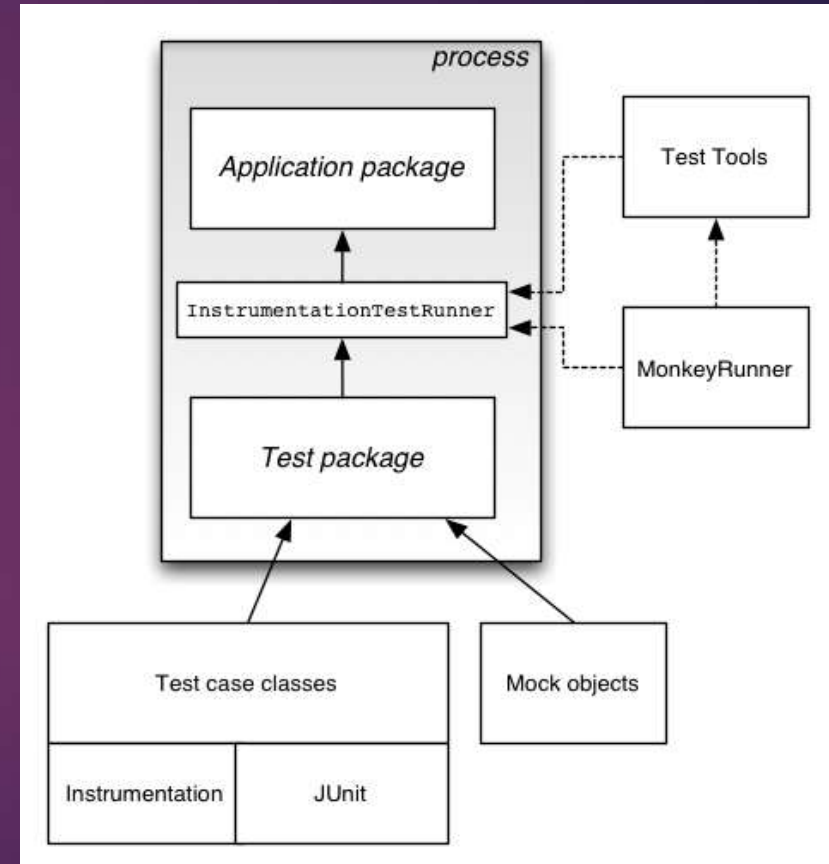
- ❑ monkeyrunner

- monkeyrunner API is contained in three modules : MonkeyRunner, MonkeyDevice and MonkeyImage
 - it does not import these modules automatically
 - To import a module, use the Python from statement:

```
from com.android.monkeyrunner import <module>
```

How does Android Test Framework works

- ❑ is based on JUnit
- ❑ test tools are used to load the test package and the application under test
- ❑ after loading package and application test tools execute an Android-specific test runner
- ❑ Test cases are run by a test runner class that loads the test case class, setups, runs, and tears down each test.
- ❑ InstrumentationTestRunner is the primary Android test runner class.



Robotium

- ❑ is an open source library extending JUnit with plenty of useful methods for Android UI testing
- ❑ require the application to be running on emulator/device
- ❑ provides powerful and robust automatic black-box test cases for Android apps (native and hybrid)
- ❑ Inherit from extension of Android class: `ActivityInstrumentationTestCase2`
- ❑ Uses “Solo” class to interact with your UI
- ❑ Robotium Recorder for capturing test output and screenshots

ONE DOES NOT SIMPLY



**USE ONLY ANDROID INSTRUMENTATION
TO TEST THEIR APPS**

Robotium

- ❑ The framework handles multiple Android activities automatically.
- ❑ Minimal time needed to write solid test cases.
- ❑ Readability of test cases is greatly improved, compared to standard instrumentation tests.
- ❑ Test cases are more robust due to the run-time binding to UI components.
- ❑ Integrates smoothly with Maven, Gradle or Ant to run tests as part of continuous integration.
- ❑ Relatively slow

Robotium

Add dependency to gradle file:

```
dependencies {  
    ...  
    androidTestCompile 'com.jayway.android.robotium:robotium-solo:5.2.1'  
}
```

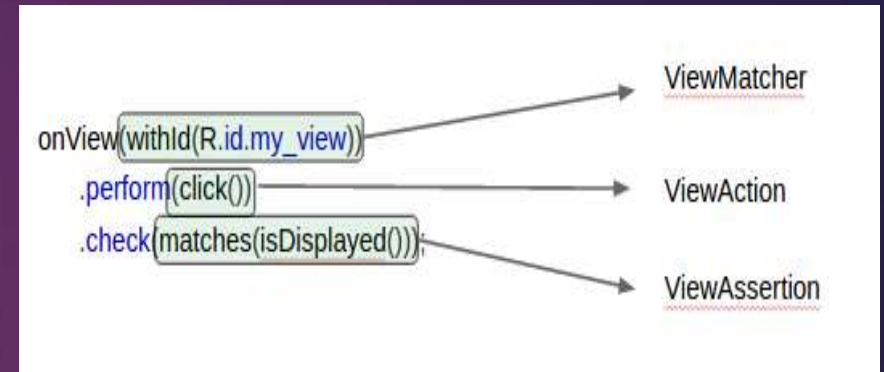
```
public class MainActivityTest extends ActivityInstrumentationTestCase2<MainActivity>{  
  
    private Solo solo;  
  
    public MainActivityTest() {  
        super(MainActivity.class);  
    }  
  
    @Override  
    public void setUp() throws Exception {  
        //setUp() is run before a test case is started.  
        //This is where the solo object is created.  
        super.setUp();  
        solo = new Solo(getInstrumentation(), getActivity());  
    }  
  
    @Override  
    public void tearDown() throws Exception {  
        //tearDown() is run after a test case has finished.  
        //finishOpenedActivities() will finish all the activities that have been opened during the test execution.  
        solo.finishOpenedActivities();  
        super.tearDown();  
    }  
  
    public void testDisplayedText() throws InterruptedException {  
        //Unlock the lock screen  
        solo.unlockScreen();  
        Assert.assertTrue(solo.searchText("Hello world,my activity"));  
    }  
}
```

Espresso

- ❑ latest Android test automation framework of Google
- ❑ API is small, predictable, easy to learn and built on top of the Android instrumentation framework
- ❑ Espresso tests can run on devices running Android 2.2 (API level 8) and higher
- ❑ we can write concise and reliable Android UI tests within a single target app
- ❑ is an instrumentation-based API and works with the AndroidJUnitRunner test runner
- ❑ Safer synchronization and thread handling
- ❑ The syntax is also a lot cleaner
- ❑ tests run optimally fast! Leave your waits, syncs, sleeps, and polls behind
- ❑ does not have support for webviews

Espresso

- ❑ Espresso is built up from 3 major components :
 - **ViewMatchers** – allows you to locate a view in the current view hierarchy [***find*** something“]
 - **ViewActions** – allows you to interact with views [***do*** something“]
 - **ViewAssertions** – allows you to assert the state of a view [***check*** something“]
- ❑ To avoid flakiness Turn off animations on testing device



Espresso

```
dependencies {  
    androidTestCompile 'com.android.support.test:runner:0.3'  
    androidTestCompile 'com.android.support.test:rules:0.3'  
    androidTestCompile 'com.android.support.test.espresso:espresso-core:2.2'  
}
```

```
@RunWith(AndroidJUnit4.class)  
public class EspressoTest {  
  
    @Rule  
    public ActivityTestRule<MainActivity> mActivityRule = new ActivityTestRule<>(MainActivity.class);  
  
    @Test  
    public void testSayHello(){  
        // locate the view with id "user_name" and type the text "John"  
        onView(withId(R.id.user_name)).perform(typeText("John"));  
  
        // locate the view with id "next" and click on it  
        onView(withId(R.id.next)).perform(click());  
  
        // locate the view with id "greeting_message" and check its text is equal with "Hello John!"  
        onView(withId(R.id.greeting_message)).check(matches(withText("Hello John!")));  
    }  
}
```

Robotium Vs. Espresso

❑ The major advances in Espresso over Robotium:

1. Synchronization.
2. API.
3. Clear failure information.

Espresso Vs. Robotium

- ❑ Espresso test run is in sync with the UI thread and does not rely on sleep/poll mechanisms but is rather event driven
 - ❑ Espresso is much faster than Robotium, but only works on some SDK versions.
 - ❑ More robust and extendable
 - ❑ Tightly coupled with Android
 - Example: ViewMatcher
 - Custom Exception Handling
 - ❑ Espresso has a small, well-defined and predictable API, which is open to customization
- ❑ Robotium attempts to address thread safety with sleep/retry mechanisms, which makes it unreliable and slower than necessary
 - ❑ Easier to use, Very stable
 - ❑ prior versions of Robotium suffered from inconsistent failure handling
 - ❑ In Robotium's API, it is expected to choose from 30+ click methods
 - ❑ Robotium exposes dangerous methods like `getCurrentActivity` and `getView`, which allow you to operate on objects outside of the main thread

uiautomator

- ❑ Can Test UI of native Android apps on one or more devices
- ❑ perform interactions on user apps and system apps
- ❑ is an instrumentation-based API and works with the AndroidJUnitRunner test runner
- ❑ Requires Android 4.3 (API level 18) or higher
- ❑ is well-suited for writing *black box*-style automated tests
- ❑ APIs interact with visible elements on a device, regardless of which Activity is in focus
- ❑ runs JUnit test cases with special privileges, which means test cases can span across different processes.
- ❑ doesn't support webview, with no way to directly access Android objects.

uiautomator

```
dependencies {  
    androidTestCompile 'com.android.support.test:runner:0.3'  
    androidTestCompile 'com.android.support.test:rules:0.3'  
    androidTestCompile 'com.android.support.test.uiautomator:uiautomator-v18:2.1.1'  
}
```

```
public class defaultLauncherUiTest extends InstrumentationTestCase {  
  
    private UiDevice mDevice;  
  
    public void setUp() {  
        // Initialize UiDevice instance  
        mDevice = UiDevice.getInstance(getInstrumentation());  
  
        // Start from the home screen  
        mDevice.pressHome();  
        // Bring up the default launcher by searching for  
        // a UI component that matches the content-description for the launcher button  
        UiObject allAppsButton = mDevice.findObject(new UiSelector().description("Apps"));  
  
        // Perform a click on the button to bring up the launcher  
        allAppsButton.clickAndWaitForNewWindow();  
    }  
}
```

Espresso Vs. uiautomator

❑ Testing UI for a Single App:

- checks that the target app returns the correct UI output in response to user interactions in the app's activities.
- Espresso framework is used to simulate user actions programmatically and test complex intra-app user interactions.

❑ Testing UI for Multiple Apps:

- verifies the correct behavior of interactions between different user apps or between user apps and system apps.
- UI Automator framework is used to support cross-app interactions.

Robolectric

- ❑ Robolectric is a unit test framework
- ❑ lets you run your tests inside the JVM
- ❑ Doesn't require emulator/Device to run test
- ❑ Fastest testing suite; reduces test cycles from minutes to seconds
- ❑ No Mocking Framework is needed
- ❑ does not work for every case and only supports unit tests
- ❑ When tests fall outside its scope then we need to rely on Robotium for complete integration testing

Robolectric

Building with Gradle

Add the following to your build.gradle:

```
testCompile "org.robolectric:robolectric:3.0"
```

Annotate your test with the Gradle test runner:

```
@RunWith(RobolectricGradleTestRunner.class)
@Config(constants = BuildConfig.class)
public class SandwichTest {
}
```

```
@RunWith(RobolectricTestRunner.class)
public class MyActivityTest {

    @Test
    public void clickingButton_shouldChangeResultsViewText() throws Exception {
        MyActivity activity = Robolectric.setupActivity(MyActivity.class);

        Button button = (Button) activity.findViewById(R.id.button);
        TextView results = (TextView) activity.findViewById(R.id.results);

        button.performClick();
        assertEquals(results.getText().toString(), "Robolectric Rocks!");
    }
}
```

Other 3rd Party Frameworks

- ❑ Selendroid

<http://selendroid.io/>

- ❑ Testdroid

<http://testdroid.com/>

- ❑ Appium

- ❑ Calabash

References

http://developer.android.com/tools/testing/testing_android.html

<https://code.google.com/p/robotium/>

<http://robolectric.org/>

https://github.com/codepath/android_guides/wiki/Android-Unit-and-Integration-testing

<http://developer.android.com/training/testing/ui-testing/espresso-testing.html>

<https://code.google.com/p/android-test-kit/wiki/Espresso>

<http://developer.android.com/tools/testing-support-library/index.html#UIAutomator>

<http://testdroid.com/tech/top-5-android-testing-frameworks-with-examples>

<https://www.youtube.com/watch?v=TGU0B4qRIHY>

<https://androidresearch.wordpress.com/2015/04/04/an-introduction-to-espresso/>

<http://www.vogella.com/tutorials/AndroidTestingEspresso/article.html>

<http://www.vogella.com/tutorials/AndroidTesting/article.html>